REWE digital Academy 2025

Medical Record Project

(Project Guide)

Stefan Velev

The project aims to develop a Patient Medical Record system with the following models:

Role, User, Person, Address, Doctor, Specialty, General Practitioner, Patient, Medical Visit, Diagnose, Treatment, Sick Leave

For each model I have created a REST controller, repository, service interface, service implementation, several data transfer objects (DTO), mappers (MapStruct).

The structure of the project relies on ***JOINED inheritance strategy*** which starts from User as base class and continues up to 3 levels to General Practitioner (General Practitioner IS-A Doctor IS-A Person IS-A User). This means that the primary keys of the subclasses are at the same time foreign keys to the superclass.

The configurations for the database are used as variables in the ***application.properties*** file. They are defined in ***.env*** file and for that reason I have included driver as a dependency to connect them. I have added ***Swagger OpenAPI documentation*** as a dependency that can be accessed when the application is started on ***localhost:8080/swagger.html***. For the developments of the project I have used **Oracle OpenJDK 19**.

I have provided a **Postman collection** with the project where all the endpoints can be tried. There is dummy data that is in **data.sql** script which can be initially loaded by uncommenting the relevant rows in the **application.properties** file for the first time.

The test users that can be used from **data.sql** for the tests (if preferred we can create our own with the relevant POST methods and they will automatically be assigned to their roles) with **Postman** are with the following roles:

ADMIN:

ID: *5* Username: *admin* Password: *admin*

DOCTOR:

ID: *1* Username: *john.doe* Password: *john.doe*

ID: *2* Username: *jane.smith* Password: *jane.smith*

PATIENT:

ID: *3* Username: *alice.jones* Password: *alice.jones*

ID: *4* Username: *bob.brown* Password: *bob.brown*

The roles in the system are: **PATIENT**, **DOCTOR** and **ADMIN**. The rights that they have related to the access to the different controllers are presented in the following table:

Method	Endpoint	ADMIN	DOCTOR	PATIENT
Address Controller				
GET	/api/addresses/{id}	✓	✓	X
GET	/api/addresses	✓	✓	X
POST	/api/addresses	✓	✓	X

PUT	/api/addresses/{id}	✓	✓	X
DELETE	/api/addresses/{id}	✓	✓	X
Person Controller				
GET	/api/persons/{id}	✓	✓	✓ (only own)
GET	/api/persons	✓	✓	X
POST	/api/persons	✓	✓	X
PUT	/api/persons/{id}	✓	✓	X
DELETE	/api/persons/{id}	✓	✓	X
Doctor Controller				
GET	/api/doctors/{id}	✓	✓ (only own)	X
GET	/api/doctors	✓	X	X
POST	/api/doctors	✓	X	X
PUT	/api/doctors/{id}	✓	✓ (only own)	X
DELETE	/api/doctors/{id}	✓	X	X
PATCH	/api/doctors/{id} (specialty manage)	✓	X	X
GET	/api/doctors/most-sick-leaves	✓	X	X
Specialty Controller				
GET	/api/specialties/{id}	✓	✓	X
GET	/api/specialties	✓	✓	X
POST	/api/specialties	✓	X	X
PUT	/api/specialties/{id}	✓	X	X
DELETE	/api/specialties/{id}	✓	X	X
General Practitioner Controller				
GET	/api/general-practitioners/{id}	✓	✓ (only own)	X
GET	/api/general-practitioners	✓	X	X
POST	/api/general-practitioners	✓	X	X
PUT	/api/general-practitioners/{id}	✓	✓ (only own)	X
DELETE	/api/general-practitioners/{id}	✓	X	X
Patient Controller				
GET	/api/patients/{id}	✓	✓	✓ (only own)
GET	/api/patients	✓	✓	X
POST	/api/patients	✓	X	X
PUT	/api/patients/{id}	✓	X	X
DELETE	/api/patients/{id}	✓	X	X
PATCH	/api/patients/{id} (GP management)	✓	✓	X
GET	/api/patients/general-practitioner/{gp_id}	✓	✓ (only own)	X
GET	/api/patients/diagnose/{diagnose_id}	✓	X	X

GET	/api/patients/count/general-practitioner	✓	X	X
GET	api/patients/count/general-practitioner/{gp_id}	✓	X	X
Medical Visit Controller				
GET	/api/medical-visits/{id}	✓	✓ (only own)	✓ (only own)
GET	/api/medical-visits	✓	X	X
POST	/api/medical-visits	✓	✓	X
PUT	/api/medical-visits/{id}	✓	✓ (only own)	X
DELETE	/api/medical-visits/{id}	✓	X	X
PATCH	/api/medical-visits/{id} (diagnose/treatment manage)	✓	✓ (only own)	X
GET	/api/medical-visits/period	✓	X	X
GET	/api/medical-visits/patient	✓	X	X
GET	/api/medical-visits/patient/{patient_id}	✓	X	X
GET	/api/medical-visits/doctor/{doctor_id}/period	✓	X	X
GET	/api/medical-visits/count/doctor	✓	X	X
GET	/api/medical-visits/count/doctor/{doctor_id}	✓	X	X
Diagnose Controller				
GET	/api/diagnoses/{id}	✓	✓	X
GET	/api/diagnoses	✓	✓	X
POST	/api/diagnoses	✓	X	X
PUT	/api/diagnoses/{id}	✓	X	X
DELETE	/api/diagnoses/{id}	✓	X	X
GET	/api/diagnoses/rank	✓	X	X
Treatment Controller				
GET	/api/treatments/{id}	✓	✓	X
GET	/api/treatments	✓	✓	X
POST	/api/treatments	✓	✓	X
PUT	/api/treatments/{id}	✓	X	X
DELETE	/api/treatments/{id}	✓	X	X
Sick Leave Controller				
GET	/api/sick-leaves/{id}	✓	✓	X
GET	/api/sick-leaves	✓	✓	X
POST	/api/sick-leaves	✓	✓	X
PUT	/api/sick-leaves/{id}	✓	✓ (only own)	X
DELETE	/api/sick-leaves/{id}	✓	X	X
GET	/api/sick-leaves/months-count-history-statistic	✓	X	X

Note: *There is a bug in the Reverse Engineer tool of MySQL Workbench that presents the one-to-one relationships as one-to-many. The exact type of the relationships can be seen described in the code.*

